WWW.SabzElco.IR

5 Power

5.1 Introduction

On Earth, apart from nuclear sources, all energy is or has been stockpiled from the sun. In essence, Earth is a huge battery that has been charged up over billions of years via the energy of sunlight in the form of plant growth, which in turn has been turned to carbon and then to oil, gas, coal or other carbon-based fuels. Additionally, in these times, we can harvest energy directly from the sun (solar power), or indirectly from the wind, tides, precipitation (hydro) or geothermal. Energy undergoes transformations. Sunlight to plant growth. Plants to carbon. Carbon to heat. Heat to electricity. Electricity to chemical (battery charging). Chemical to electricity (battery discharging). Electricity to audio (playing an MP3). In the last conversion, some energy is transformed into sound that dissipates into the universe. The rest is turned to heat as the tunes are decoded and played. It is also lost to the universe (perhaps warming our hands slightly on a cold night). So pervasive are energy transformations in everyday life, we are often not at all aware of them. Most times they occur quietly and unnoticed.

Today, we are interested in power from a number of points of view. In portable applications, products normally run off batteries. While battery technology has improved markedly over the years, it remains that a battery of a certain weight and size has a certain energy capacity. For example, a pair of rechargeable AA batteries has an energy capacity of about 7 W-hr, and a good lithium-ion laptop battery has an energy density of about 80 W-hr/lb. Inevitably, the battery runs down and needs recharging or replacement. Product designers are interested in extending the lifetime of the battery while simultaneously adding features and reducing size, so creating low-power IC designs is key. In applications that are permanently connected to a power cord, the ever-present need to reduce dependence on fossil fuels and reduce greenhouse emissions leads us to look for low power solutions to all problems involving electronics. High-performance chips are limited to about 150 W before liquid cooling or other costly heat sinks become necessary. In 2006, data centers and servers in the United States consumed 61 billion kWh of electricity [EPA07]. This represents the output of 15 power plants, costs about $4.5 billion, and amounts to 1.5% of total U.S. energy consumption—more than that consumed by all the television sets in the country. While chip functionality was once limited by area, it is now often constrained by power. High-performance design and energy-efficient design have become synonymous.

In this chapter, we will examine the fundamental theory behind the various sources of power dissipation in a CMOS chip. Next, we will look at methods of estimating and minimizing these sources. Then, some architectural ideas for achieving low power are discussed.

While we concentrate mainly on the methods available as an IC designer to reduce power, it should be remembered that it is the application and architectural level where the major impact on power dissipation can be made. Quite simply stated, the less time you have a circuit turned on, the less power it will dissipate. It is a simple maxim, but drives all of the work on extremely low power circuits. To state this again, you must optimize power in a top-down manner, from the problem definition downward. Do not optimize from the bottom up, i.e., the circuit level; you will be doomed to fail.

5.1.1 Definitions

We have thrown some terms about already including power and energy. It is informative to go back to basics and examine what we mean by these terms and why we are even interested in them.

The *instantaneous power* $P(t)$ consumed or supplied by a circuit element is the product of the current through the element and the voltage across the element

$$P(t) = I(t)V(t) \tag{5.1}$$

The *energy* consumed or supplied over some time interval T is the integral of the instantaneous power

$$E = \int_0^T P(t)dt \tag{5.2}$$

The *average power* over this interval is

$$P_{\text{avg}} = \frac{E}{T} = \frac{1}{T}\int_0^T P(t)dt \tag{5.3}$$

Power is expressed in units of Watts (W). Energy in circuits is usually expressed in Joules (J), where 1 W = 1 J/s. Energy in batteries is often given in W-hr, where 1 W-hr = (1 J/s)(3600 s/hr)(1 hr) = 3600 J.



FIGURE 5.1
Resistor

5.1.2 Examples

Figure 5.1 shows a resistor. The voltage and current are related by Ohm's Law, $V = IR$, so the instantaneous power dissipated in the resistor is

$$P_R(t) = \frac{V_R^2(t)}{R} = I_R^2(t)R \tag{5.4}$$

This power is converted from electricity to heat.

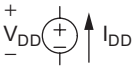


FIGURE 5.2
Voltage source

Figure 5.2 shows a voltage source V_{DD}. It supplies power proportional to its current

$$P_{VDD}(t) = I_{DD}(t)V_{DD} \tag{5.5}$$

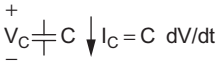


FIGURE 5.3
Capacitor

Figure 5.3 shows a capacitor. When the capacitor is charged from 0 to V_C, it stores energy E_C

$$E_C = \int_0^\infty I(t)V(t)dt = \int_0^\infty C\frac{dV}{dt}V(t)dt = C\int_0^{V_C} V(t)dV = \tfrac{1}{2}CV_C^2 \tag{5.6}$$

The capacitor releases this energy when it discharges back to 0.

Figure 5.4 shows a CMOS inverter driving a load capacitance. When the input switches from 1 to 0, the pMOS transistor turns ON and charges the load to V_{DD}. According to EQ (5.6), the energy stored in the capacitor is

$$E_C = \tfrac{1}{2} C_L V_{DD}^2 \tag{5.7}$$

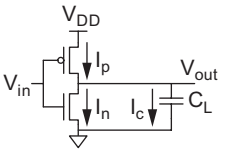


FIGURE 5.4
CMOS inverter

The energy delivered from the power supply is

$$E_C = \int_0^\infty I(t) V_{DD}\, dt = \int_0^\infty C \frac{dV}{dt} V_{DD}\, dt = C V_{DD} \int_0^{V_{DD}} dV = C V_{DD}^2 \tag{5.8}$$

Observe that only half of the energy from the power supply is stored in the capacitor. The other half is dissipated (converted to heat) in the pMOS transistor because the transistor has a voltage across it at the same time a current flows through it. The power dissipated depends only on the load capacitance, not on the size of the transistor or the speed at which the gate switches. Figure 5.5 shows the energy and power of the supply and capacitor as the gate switches.

When the input switches from 0 back to 1, the pMOS transistor turns OFF and the nMOS transistor turns ON, discharging the capacitor. The energy stored in the capacitor is dissipated in the nMOS transistor. No energy is drawn from the power supply during this transition. The same analysis applies for any static CMOS gate driving a capacitive load.

Figure 5.5 shows the waveforms as the inverter drives a 150 fF capacitor at 1 GHz. When V_{in} begins to fall, the pMOS transistor starts to turn ON. It is initially saturated, and the current I_p ramps up and eventually levels out at I_{dsat} as V_{in} falls. Eventually, V_{out} rises to the point that the pMOS shifts to the linear regime. I_p tapers off exponentially, as

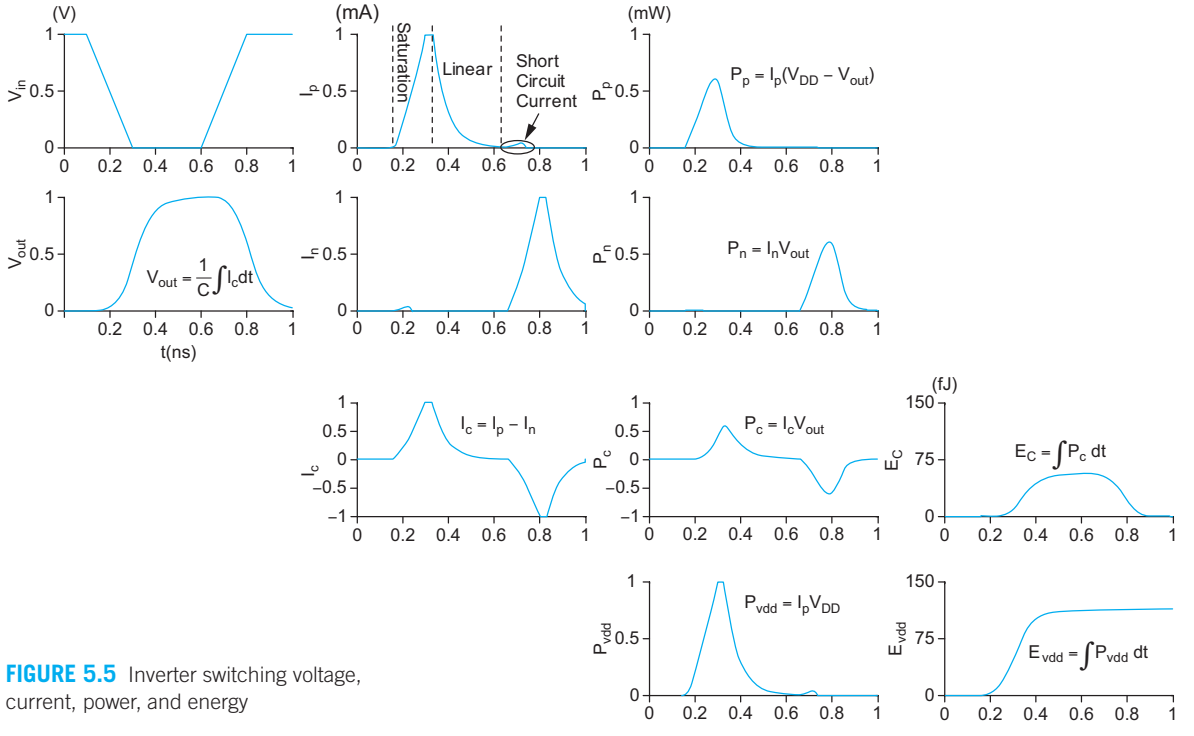


FIGURE 5.5 Inverter switching voltage, current, power, and energy

one would expect charging a capacitor through a linear resistor. When V_{in} rises, the pMOS starts to turn OFF. However, there is a small blip of current while the partially ON pMOS fights against the nMOS. This is called *short-circuit current*. The inverter draws power from V_{DD} as V_{out} rises. Half of the power is dissipated in the pMOS transistor and the other half is delivered to the capacitor. V_{DD} supplies a total of 150 fJ of energy, of which half is stored on the capacitor. The inverter is sized for equal rise/fall times so the falling transition is symmetric. The energy on the capacitor is dumped to GND. The short-circuit current consumes an almost imperceptibly small 2.7 fJ of additional energy from V_{DD} during this transition.

Suppose that the gate switches at some average frequency f_{sw}. Over some interval T, the load will be charged and discharged Tf_{sw} times. Then, according to EQ (5.3), the average power dissipation is

$$P_{switching} = \frac{E}{T} = \frac{Tf_{sw}CV_{DD}^2}{T} = CV_{DD}^2 f_{sw} \tag{5.9}$$

This is called the *dynamic power* because it arises from the switching of the load. Because most gates do not switch every clock cycle, it is often more convenient to express switching frequency f_{sw} as an *activity factor* α times the clock frequency f. Now, the dynamic power dissipation may be rewritten as

$$P_{switching} = \alpha CV_{DD}^2 f \tag{5.10}$$

The activity factor is the probability that the circuit node transitions from 0 to 1, because that is the only time the circuit consumes power. A clock has an activity factor of $\alpha = 1$ because it rises and falls every cycle. Most data has a maximum activity factor of 0.5 because it transitions only once each cycle. Truly random data has an activity factor of 0.25 because it transitions every other cycle. Static CMOS logic has been empirically determined to have activity factors closer to 0.1 because some gates maintain one output state more often than another and because real data inputs to some portions of a system often remain constant from one cycle to the next.

5.1.3 Sources of Power Dissipation

Power dissipation in CMOS circuits comes from two components:

- Dynamic dissipation due to
  - charging and discharging load capacitances as gates switch
  - "short-circuit" current while both pMOS and nMOS stacks are partially ON
- Static dissipation due to
  - subthreshold leakage through OFF transistors
  - gate leakage through gate dielectric
  - junction leakage from source/drain diffusions
  - contention current in ratioed circuits (see Section 9.2.2)

Putting this together gives the total power of a circuit

$$P_{dynamic} = P_{switching} + P_{short\ circuit} \tag{5.11}$$

$$P_{\text{static}} = \left(I_{\text{sub}} + I_{\text{gate}} + I_{\text{junct}} + I_{\text{contention}}\right)V_{DD} \quad \textbf{(5.12)}$$

$$P_{\text{total}} = P_{\text{dynamic}} + P_{\text{static}} \quad \textbf{(5.13)}$$

Power can also be considered in active, standby, and sleep modes. *Active power* is the power consumed while the chip is doing useful work. It is usually dominated by $P_{\text{switching}}$. *Standby power* is the power consumed while the chip is idle. If clocks are stopped and ratioed circuits are disabled, the standby power is set by leakage. In sleep mode, the supplies to unneeded circuits are turned off to eliminate leakage. This drastically reduces the *sleep power* required, but the chip requires time and energy to wake up so sleeping is only viable if the chip will idle for long enough.

[Gonzalez96] found that roughly one-third of microprocessor power is spent on the clock, another third on memories, and the remaining third on logic and wires. In nanometer technologies, nearly one-third of the power is leakage. High-speed I/O contributes a growing component too. For example, Figure 5.6 shows the active power consumption of Sun's 8-core 84 W Niagra2 processor [Nawathe08]. The cores and other components collectively account for clock, logic, and wires.

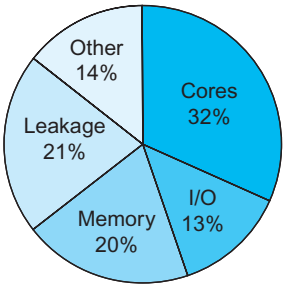


FIGURE 5.6 Power in Niagra2

The next sections investigate how to estimate and minimize each of these components of power. Many tools are available to assist with power estimation; these are discussed further in Sections 8.5.4 and 14.4.1.6.

5.2 Dynamic Power

Dynamic power consists mostly of the switching power, given in EQ (5.10). The supply voltage V_{DD} and frequency f are readily known by the designer. To estimate this power, one can consider each node of the circuit. The capacitance of the node is the sum of the gate, diffusion, and wire capacitances on the node. The activity factor can be estimated using techniques described in Section 5.2.1 or measured from logic simulations. The *effective capacitance* of the node is its true capacitance multiplied by the activity factor. The switching power depends on the sum of the effective capacitances of all the nodes.

Activity factors can be heavily dependent on the particular task being executed. For example, a processor in a cell phone will use more power while running video games than while displaying a calendar. CAD tools do a fine job of power estimation when given a realistic workload. Low power design involves considering and reducing each of the terms in switching power.

As V_{DD} is a quadratic term, it is good to select the minimum V_{DD} that can support the required frequency of operation. Likewise, we choose the lowest frequency of operation that achieves the desired end performance. The activity factor is mainly reduced by putting unused blocks to sleep. Finally, the circuit may be optimized to reduce the overall load capacitance of each section.

Example 5.1

A digital system-on-chip in a 1 V 65 nm process (with 50 nm drawn channel lengths and λ = 25 nm) has 1 billion transistors, of which 50 million are in logic gates and the remainder in memory arrays. The average logic transistor width is 12 λ and the average memory transistor width is 4 λ. The memory arrays are divided into banks and only the

necessary bank is activated so the memory activity factor is 0.02. The static CMOS logic gates have an average activity factor of 0.1. Assume each transistor contributes 1 fF/μm of gate capacitance and 0.8 fF/μm of diffusion capacitance. Neglect wire capacitance for now (though it could account for a large fraction of total power). Estimate the switching power when operating at 1 GHz.

SOLUTION: There are $(50 \times 10^6$ logic transistors$)(12\ \lambda)(0.025\ \mu m/\lambda)((1 + 0.8)$ fF/μm) = 27 nF of logic transistors and $(950 \times 10^6$ memory transistors$)(4\ \lambda)(0.025\ \mu m/\lambda)((1 + 0.8)$ fF/μm) = 171 nF of memory transistors. The switching power consumption is $[(0.1)(27 \times 10^{-9}) + (0.02)(171 \times 10^{-9})](1.0\ V)^2(10^9\ Hz) = 6.1$ W.

Dynamic power also includes a short-circuit power component caused by power rushing from V_{DD} to GND when both the pullup and pulldown networks are partially ON while a transistor switches. This is normally less than 10% of the whole, so it can be conservatively estimated by adding 10% to the switching power.

Switching power is consumed by delivering energy to charge a load capacitance, then dumping this energy to GND. Intuitively, one might expect that power could be saved by shuffling the energy around to where it is needed rather than just dumping it. Resonant circuits, and adiabatic charge-recovering circuits [Maksimovic00, Sathe07] seek to achieve such a goal. Unfortunately, all of these techniques add complexity that detracts from the potential energy savings, and none have found more than niche applications.

5.2.1 Activity Factor

The activity factor is a powerful and easy-to-use lever for reducing power. If a circuit can be turned off entirely, the activity factor and dynamic power go to zero. Blocks are typically turned off by stopping the clock; this is called *clock gating*. When a block is on, the activity factor is 1 for clocks and substantially lower for nodes in logic circuits. The activity factor of a logic gate can be estimated by calculating the switching probability. Glitches can increase the activity factor.

5.2.1.1 Clock Gating Clock gating ANDs a clock signal with an enable to turn off the clock to idle blocks. It is highly effective because the clock has such a high activity factor, and because gating the clock to the input registers of a block prevents the registers from switching and thus stops all the activity in the downstream combinational logic.

Clock gating can be employed on any enabled register. Section 10.3.5 discusses enabled register design. Sometimes the logic to compute the enable signal is easy; for example, a floating-point unit can be turned off when no floating-point instructions are being issued. Often, however, clock gating signals are some of the most critical paths of the chip.

The clock enable must be stable while the clock is active (i.e., 1 for systems using positive edge-triggered flip-flops). Figure 5.7 shows how an enable latch can be used to ensure the enable does not change before the clock falls.

When a large block of logic is turned off, the clock can be gated early in the clock distribution network, turning off not only the registers but also a portion of the global network. The clock network has an activity factor of 1 and a high capacitance, so this saves significant power.

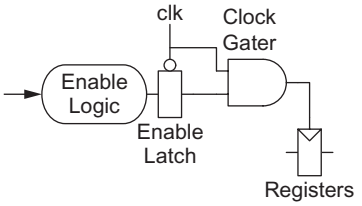


FIGURE 5.7 Clock gating

5.2.1.2 Switching Probability Recall that the activity factor of a node is the probability that it switches from 0 to 1. This probability depends on the logic function. By analyzing the probability that each node is 1, we can estimate the activity factors. Although designers don't manually estimate activity factors very often, the exercise is worth doing here to gain some intuition about switching activity.

Define P_i to be the probability that node i is 1. $\overline{P}_i = 1 - P_i$ is the probability that node i is 0. α_i, the activity factor of node i, is the probability that the node is 0 on one cycle and 1 on the next. If the probability is uncorrelated from cycle to cycle,

$$\alpha_i = \overline{P}_i P_i \tag{5.14}$$

Completely random data has $P = 0.5$ and thus $\alpha = 0.25$. Structured data may have different probabilities. For example, the upper bits of a 64-bit unsigned integer representing a physical quantity such as the intensity of a sound or the amount of money in your bank account are 0 most of the time. The activity factor is lower than 0.25 for such data.

Table 5.1 lists the output probabilities of various gates as a function of their input probabilities, assuming the inputs are uncorrelated. According to EQ (5.14), the activity factor of the output is $\overline{P}_Y P_Y$.

TABLE 5.1 Switching probabilities

Gate	P_Y
AND2	$P_A P_B$
AND3	$P_A P_B P_C$
OR2	$1 - \overline{P}_A \overline{P}_B$
NAND2	$1 - P_A P_B$
NOR2	$\overline{P}_A \overline{P}_B$
XOR2	$P_A \overline{P}_B + \overline{P}_A P_B$

Example 5.2

Figure 5.8 shows a 4-input AND gate built using a tree (a) and a chain (b) of gates. Determine the activity factors at each node in the circuit assuming the input probabilities $P_A = P_B = P_C = P_D = 0.5$.

SOLUTION: Figure 5.9 labels the signal probabilities and the activity factors at each node based on Table 5.1 and EQ (5.14). The chain has a lower activity factor at the intermediate nodes.

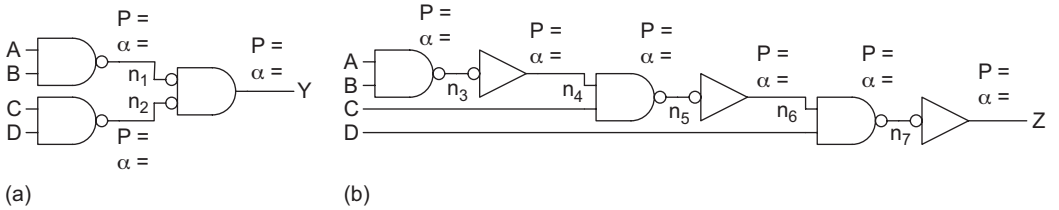


FIGURE 5.8 4-input AND circuits

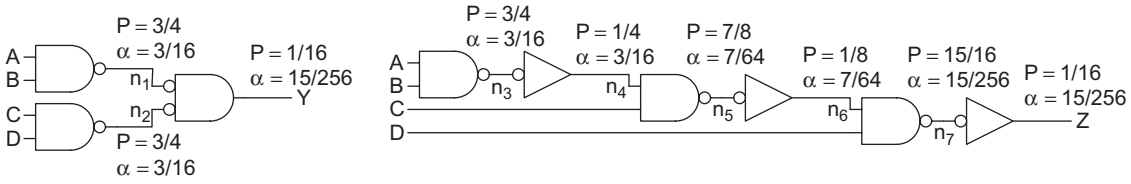


FIGURE 5.9 Signal probabilities and activity factors

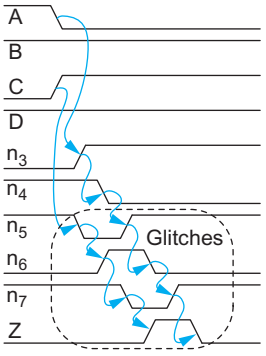


FIGURE 5.10
Glitching in a chain of gates

When paths contain reconvergent fanouts, signals become correlated and conditional probabilities become required. Power analysis tools are the most convenient way to handle large complex circuits.

Preliminary power estimation requires guessing an activity factor before RTL code is written and workloads are known. $\alpha = 0.1$ is a reasonable choice in the absence of better data.

5.2.1.3 Glitches The switching probabilities computed in the previous section are only valid if the gates have zero propagation delay. In reality, gates sometimes make spurious transitions called glitches when inputs do not arrive simultaneously. For example, in Figure 5.8(b), suppose *ABCD* changes from 1101 to 0111. Node n_4 was 1 and falls to 0. However, nodes n_5, n_6, n_7, and Z may glitch before n_4 changes, as shown in Figure 5.10. The glitches cause extra power dissipation. Chains of gates are particularly prone to this problem. Glitching can raise the activity factor of a gate above 1 and can account for the majority of power in certain circuits such as ripple carry adders and array multipliers (see Chapter 11). Glitching power can be accurately assessed through simulations accounting for timing.

5.2.2 Capacitance

Switching capacitance comes from the wires and transistors in a circuit. Wire capacitance is minimized through good floorplanning and placement (the locality aspect of structured design). Units that exchange a large amount of data should be placed close to each other to reduce wire lengths.

Device-switching capacitance is reduced by choosing fewer stages of logic and smaller transistors. Minimum-sized gates can be used on non-critical paths. Although Logical Effort finds that the best stage effort is about 4, using a larger stage effort increases delay only slightly and greatly reduces transistor sizes. Therefore, gates that are large or have a high activity factor and thus dominate the power can be downsized with only a small performance impact. For example, buffers driving I/O pads or long wires may use a stage effort of 8–12 to reduce the buffer size. Similarly, registers should use small clocked transistors because their activity factor is an order of magnitude greater than transistors in combinational logic. In Chapter 6, we will see that wire capacitance dominates many circuits. The most energy-efficient way to drive long wires is with inverters or buffers rather than with more complex gates that have higher logical efforts [Stan99].

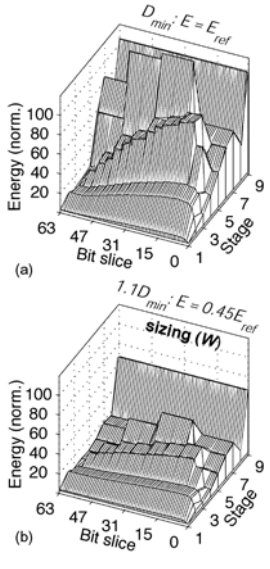


FIGURE 5.11 Adder gate sizing under a delay constraint (Adapted from [Marković04]. © IEEE 2004.)

Figure 5.11 shows an example of transistor sizing in a 64-bit Kogge-Stone adder (see Section 11.2.2.8) [Marković04]. In Figure 5.11(a), the gates are sized to achieve minimum possible delay. The high spikes in the middle correspond to large gates driving the long wires. In Figure 5.11(b), the circuit is reoptimized for 10% greater delay. The energy is reduced by 55%. In general, large energy savings can be made by relaxing a circuit a small amount from the minimum delay point.

Unfortunately, there are no closed-form methods to determine gate sizes that minimize energy under a delay constraint, even for circuits as simple as an inverter chain

[Ma94]. However, it is straightforward to solve the problem numerically, as will be formulated in the next section.

5.2.2.1 Gate Sizing Under a Delay Constraint In Chapter 4, Logical Effort showed us how to minimize delay. In many cases, we are willing to increase delay to save energy. We can extend the iterative technique from Section 4.5.6 to size a circuit for minimum switching energy under a delay constraint.



First, consider a model to compute the energy of a circuit. If a unit inverter has gate capacitance $3C$, then a gate with logical effort g, parasitic delay p, and drive x has gx times as much gate capacitance and px times as much diffusion capacitance. The switching energy of each gate depends on its activity factor, the diffusion capacitance of the gate, the wire capacitance C_{wire}, and the gate capacitance of all the stages it drives. The energy of the entire circuit is the sum of the energies of each gate.

$$\text{Energy} = 3CV_{DD}^2 \sum_{i\in\text{nodes}} \alpha_i \left(\frac{C_{\text{wire}_i}}{3C} + p_i x_i + \sum_{j\in\text{fanout}(i)} g_j x_j \right) \tag{5.15}$$

If wire capacitance is expressed in multiples of the capacitance of a unit inverter as $c = C_{\text{wire}}/3C$ and we normalize energy for the capacitance and voltage of the process, EQ (5.15) becomes the sum of the effective capacitances of the nodes.

$$E = \sum_{i\in\text{nodes}} \alpha_i \left(c_i + p_i x_i + \sum_{j\in\text{fanout}(i)} g_j x_j \right) = \sum_{i\in\text{nodes}} \alpha_i x_i d_i \tag{5.16}$$

Now, we seek to minimize E such that the worst-case arrival time is less than some delay D. The problem is still a posynomial and has a unique solution that can be found quickly by a good optimizer.

Example 5.3

Generate an energy-delay trade-off curve for the circuit from Figure 4.37 as delay varies from the minimum possible ($D_{\min} = 23.44\ \tau$) to 50 τ. Assume that the input probabilities are 0.5.

SOLUTION: Figure 5.12 shows the activity factors of each node. Hence, the energy of this circuit is

$$\begin{aligned} E = {} & \tfrac{1}{4}\left(1 + \tfrac{4}{3}x_2 + \tfrac{5}{3}x_3\right) + \tfrac{3}{16}\left(2x_2 + \tfrac{7}{3}x_4\right) + \tfrac{3}{16}\left(2x_3 + \tfrac{7}{3}x_4\right) \\ & + \tfrac{87}{1024}\left(10 + 3x_4 + x_5\right) + \tfrac{87}{1024}\left(12 + x_5\right) \end{aligned} \tag{5.17}$$

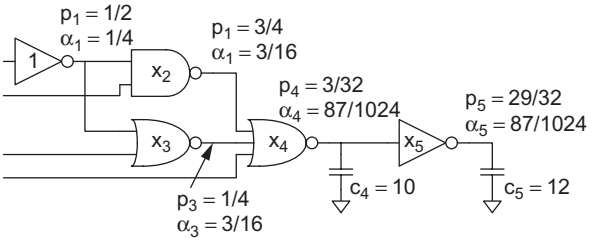


FIGURE 5.12 Activity factors

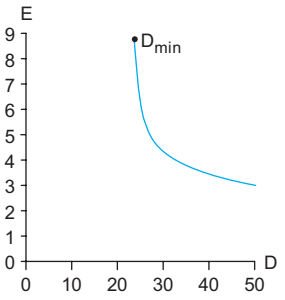


FIGURE 5.13
Energy-delay trade-off curve

Figure 5.13 shows the energy-delay trade-off curve obtained by repeatedly solving for minimum energy under a delay constraint. The curve is steep near D_{min}, indicating that a large amount of energy can be saved for a small increase in delay. The delay cannot be reduced below D_{min} for any amount of energy unless the size of the input inverter is increased (which would increase the delay of the previous circuit).

5.2.3 Voltage

Voltage has a quadratic effect on dynamic power. Therefore, choosing a lower power supply significantly reduces power consumption. As many transistors are operating in a velocity-saturated regime, the lower power supply may not reduce performance as much as long-channel models predict. The chip may be divided into multiple *voltage domains*, where each domain is optimized for the needs of certain circuits. For example, a system-on-chip might use a high supply voltage for memories to ensure cell stability, a medium voltage for a processor, and a low voltage for I/O peripherals running at lower speeds. In Section 5.3.2, we will examine how voltage domains can be turned off entirely to save leakage power during sleep mode.

Voltage also can be adjusted based on operating mode; for example, a laptop processor may operate at high voltage and high speed when plugged into an AC adapter, but at lower voltage and speed when on battery power. If the frequency and voltage scale down in proportion, a cubic reduction in power is achieved. For example, the laptop processor may scale back to 2/3 frequency and voltage to save 70% in power when unplugged.

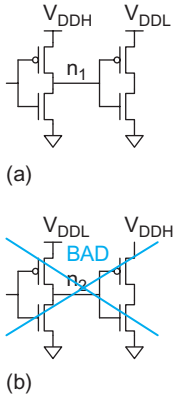


FIGURE 5.14
Voltage domain crossing

5.2.3.1 Voltage Domains Some of the challenges in using voltage domains include converting voltage levels for signals that cross domains, selecting which circuits belong in which domain, and routing power supplies to multiple domains.

Figure 5.14 shows direct connection of inverters in two domains using high and low supplies, V_{DDH} and V_{DDL}, respectively. A gate in the V_{DDH} domain can directly drive a gate in the V_{DDL} domain. However, the gate in the V_{DDL} domain will switch faster than it would if driven by another V_{DDL} gate. The timing analyzer must consider this when computing the contamination delay, lest a hold time be violated. Unfortunately, the gate in the V_{DDL} domain cannot directly drive a gate in the V_{DDH} domain. When n_2 is at V_{DDL}, the pMOS transistor in the V_{DDH} domain has $V_{gs} = V_{DDH} - V_{DDL}$. If this exceeds V_t, the pMOS will turn ON and burn contention current. Even if the difference is less than V_t, the pMOS will suffer substantially increased leakage. This problem may be alleviated by using a high-V_t pMOS device in the receiver if the voltage difference between domains is small enough [Tawfik09].

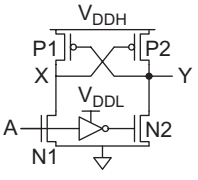


FIGURE 5.15 Level converter

The standard method to handle voltage domain crossings is a *level converter*, shown in Figure 5.15. When $A = 0$, $N1$ is OFF and $N2$ is ON. $N2$ pulls Y down to 0, which turns on $P1$, pulling X up to V_{DDH} and ensuring that $P2$ turns OFF. When $A = 1$, $N1$ is ON and $N2$ is OFF. $N1$ pulls X down to 0, which turns on $P2$, pulling Y up to V_{DDH}. In either case, the level converter behaves as a buffer and properly drives Y between 0 and V_{DDH} without risk of transistors remaining partially ON. Unfortunately, the level converter costs delay (about 2 FO4) and power at each domain crossing. [Kulkarni04] and [Ishihara04] survey a variety of other level converters. The cost can be partially alleviated by building the converter into a register and only crossing voltage domains on clock cycle boundaries. Such level-converter flops are described in Section 10.4.4.

The easiest way to use voltage domains is to associate each domain with a large area of the floorplan. Thus, each domain receives its own power grid. Note that level converters

require two power supplies, so they should be placed near the periphery of the domain where necessary for domain crossings.

An alterative approach is called *clustered voltage scaling* (CVS) [Usami95], in which two supply voltages can be used in a single block. Figure 5.16 shows an example of clustered voltage scaling. Gates early in the path use V_{DDH}. Noncritical gates later in the path use V_{DDL}. Voltages are assigned such that a path never crosses from a V_{DDL} gate to a V_{DDH} gate within a block of combinational logic, so level converters are only required at the registers. CVS requires that two power supplies be distributed across the entire block. This can be done by using two power rails. A cell library can have high- and low-voltage versions of each cell, differentiated only by the rail to which the pMOS transistors are connected, so that the flavor of gate can be interchanged. Note that many processes require a large spacing between n-wells at different potentials, which limits the proximity of the V_{DDH} and V_{DDL} gates.

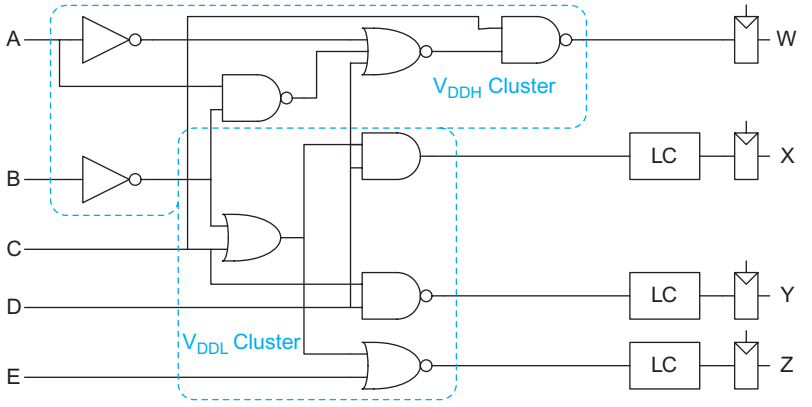


FIGURE 5.16 Clustered voltage scaling

5.2.3.2 Dynamic Voltage Scaling (DVS) Many systems have time-varying performance requirements. For example, a video decoder requires more computation for rapidly moving scenes than for static scenes. A workstation requires more performance when running SPICE than when running Solitaire. Such systems can save large amounts of energy by reducing the clock frequency to the minimum sufficient to complete the task on schedule, then reducing the supply voltage to the minimum necessary to operate at that frequency. This is called *dynamic voltage scaling* (DVS) or *dynamic voltage/frequency scaling* (DVFS) [Burd00]. Figure 5.17 shows a block diagram for a basic DVS system. The DVS controller takes information from the system about the workload and/or the die temperature. It determines the supply voltage and clock frequency sufficient to complete the workload on schedule or to maximize performance without overheating. A switching voltage regulator efficiently steps down V_{in} from a high value to the necessary V_{DD}. The core logic contains a phase-locked loop or other clock synthesizer to generate the specified clock frequency.

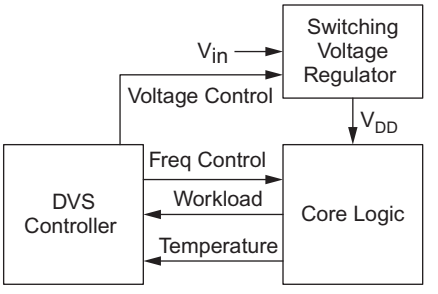


FIGURE 5.17 DVS system

The DVS controller determines the operating frequency, then chooses the lowest supply voltage suitable for that frequency. One method of choosing voltage is with a precharacterized table of voltage vs. frequency. This is inherently conservative because the voltage

should be high enough to suffice for even worst-case parts (see Chapter 7 about variability). The quad-core Itanium processor contains a fuse-programmable table that can be tailored to each chip during production [Stackhouse09]. Another method is to use a replica circuit such as a ring oscillator that tracks the performance of the system, as discussed in Section 7.5.3.4.

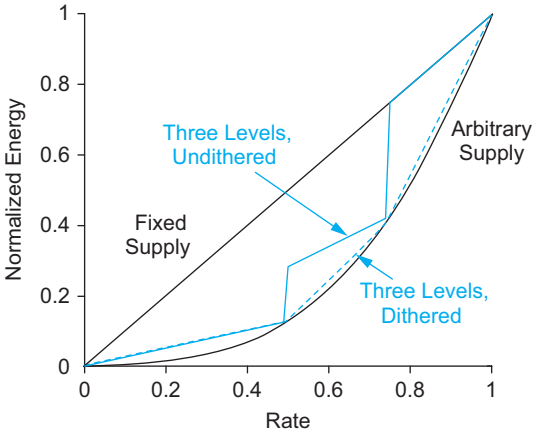


FIGURE 5.18 Energy reduction from DVS

Consider how the energy for a system varies with the workload. Define the *rate* to be the fraction of maximum performance required to complete the workload in a specified amount of time. Figure 5.18 plots energy against rate. If the rate is less than 1, the clock frequency can be adjusted down accordingly, or the system can run at full frequency until the work is done, then stop the clock and go to sleep; this may be simpler than building a continuously adjustable clock. Without DVS, the energy varies linearly with the rate. With ideal DVS, the voltage could also be reduced at lower rates. Assuming a linear relationship between voltage and frequency, the energy is proportional to the rate cubed, giving much greater savings at lower rates. Operating at half the maximum rate costs only one-eighth of the energy.

Such scaling assumes a continuously adjustable supply voltage, which is more expensive than a supply with discrete levels. Characterizing a circuit across a continuous range of voltages and frequencies is also difficult. If the supply voltage is limited to three levels, e.g., 1.0, 0.75, and 0.5 V, and the frequencies limited to three settings as well, much of the benefit of DVS still can be achieved. Better yet, a system can *dither* between these voltages to save even more energy [Gutnik97]. For example, if a rate of 0.6 is required, the system could operate at a rate of 0.75 for 40% of the computation, then switch to a rate of 0.5 for the remaining 60%. Hence, by dithering between three levels, the system can achieve almost as low energy as by using an arbitrary supply voltage. Indeed, dithering between only two supply voltages selected for full and half-rate operation is sufficient to get more than 80% of the benefit of DVS [Aisaka02].

Section 5.3.2 discusses power gating to turn off power to a block during sleep mode. The same mechanism can be used to select from one of several supply voltages for each block during active mode. This allows *local voltage dithering* so that each block can operate at a preferred voltage.

DVS normally operates over a range from the maximum V_{DD} down to about half that value. It can be extended further into the subthreshold regime [Zhai05a, Calhoun06a]; this is sometimes called *ultra-dynamic voltage scaling* (UDVS). It can be challenging to build a replica circuit that tracks the worst case delay on the chip across a very wide range of voltages. DVS is now widely used in systems ranging from consumer electronics to high-performance microprocessors [Keating07, Stackhouse09].

Subthreshold and gate leakage are strongly sensitive to the supply voltage, so DVS also is effective at reducing leakage during periods of low activity.

Operating at varied V_{DD} voltages implies an adjustable voltage regulator that reduces the voltage from a higher supply. Be careful to use a switching type regulator; otherwise, the power will just be dissipated in the regulator.

5.2.4 Frequency

Dynamic power is directly proportional to frequency, so a chip obviously should not run faster than necessary. As mentioned earlier, reducing the frequency also allows downsizing

transistors or using a lower supply voltage, which has an even greater impact on power. The performance can be recouped through parallelism (see Section 5.5.2), especially if area is not as important as power.

Even if multiple voltage supplies are not available, a chip may still use multiple frequency domains so that certain portions can run more slowly than others. For example, a microprocessor bus interface usually runs much slower than the core. Low frequency domains can also save energy by using smaller transistors.

Frequency domain crossings are easiest if the frequencies are related by integer multiples and the clocks are synchronized between domains. Section 10.6 discusses synchronization further.

5.2.5 Short-Circuit Current

Short-circuit power dissipation occurs as both pullup and pulldown networks are partially ON while the input switches, as was illustrated in Figure 5.5. It increases as the input edge rates become slower because both networks are ON for more time [Veendrick84]. However, it decreases as load capacitance increases because with large loads the output only switches a small amount during the input transition, leading to a small V_{ds} across the transistor that is causing the short-circuit current. Unless the input edge rate is much slower than the output edge rate, short-circuit current is a small fraction (< 10%) of current to the load and can be ignored in hand calculations. It is good to use relatively crisp edge rates at the inputs to gates with wide transistors to minimize their short-circuit current. This is achieved by keeping the stage effort of the previous stage reasonable, e.g., 4 or less. In general, gates with balanced input and output edge rates have low short-circuit power.

Short-circuit power is strongly sensitive to the ratio $v = V_t / V_{DD}$. In the limit that $v > 0.5$, short-circuit current is eliminated entirely because the pullup and pulldown networks are never simultaneously ON. For $v = 0.3$ or 0.2, short-circuit power is typically about 2% or 10% of switching power, respectively, assuming clean edges [Nose00a]. In nanometer processes, V_t can scarcely fall below 0.3 V without excessive leakage, and V_{DD} is on the order of 1 V, so short-circuit current has become almost negligible.

5.2.6 Resonant Circuits

Resonant circuits seek to reduce switching power consumption by letting energy slosh back and forth between storage elements such as capacitors and inductors rather than dumping the energy to ground. The technique is best suited to applications such as clocks that operate at a constant frequency.

Figure 5.19 shows a model of a resonant clock network [Chan05]. C_{clock} is the capacitance of the clock network. In an ordinary clock circuit, it is driven between V_{DD} and GND by a strong clock buffer. The resonant clock network adds the inductor L_1 and the capacitor C_2, which is approximately $10C_{clock}$. R_{clock} and R_{ind} represent losses in the clock wires and in the inductor that lower the quality of the resonator. In the resonant clock circuit, energy moves back and forth between L_1 and C_{clock}, causing a sinusoidal oscillation at the resonant frequency f. The driver pumps in just enough energy to compensate for the resistive losses. C_2 must be large enough to store excess energy and not interfere with the resonance of the clock capacitance.

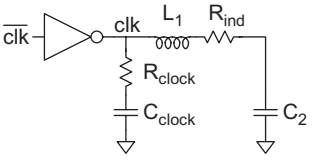


FIGURE 5.19
Resonant clock network

$$f \approx \frac{1}{2\pi\sqrt{L_1 C_{clock}}} \qquad (5.18)$$

In a mechanical analogy, inductors represent springs and capacitors represent mass. The clock itself has high capacitance and little inductance, representing a rigid mass suspended on a set of springs corresponding to the inductor L_1. The mass oscillates up and down. The clock driver gives the mass a kick to get it started and compensate for damping in the springs, but little energy is required because the springs do most of the work storing energy on the way down and delivering it back to the mass on the way up.

IBM has demonstrated a resonant global clock distribution system for the Cell processor [Chan09]. At an operating frequency of 4–5 GHz, the system could reduce chip power by 10%. Some of the drawbacks of resonant clocking include the limited range of operating frequencies, the sinusoidal clock output, and the difficulty of building a high-quality inductor in a CMOS process.

5.3 Static Power

Static power is consumed even when a chip is not switching. CMOS has replaced nMOS processes because contention current inherent to nMOS logic limited the number of transistors that could be integrated on one chip. Static CMOS gates have no contention current. Prior to the 90 nm node, leakage power was of concern primarily during sleep mode because it was negligible compared to dynamic power. In nanometer processes with low threshold voltages and thin gate oxides, leakage can account for as much as a third of total active power. Section 2.4.4 introduced leakage current mechanisms. This section briefly reviews each source of static power. It then discusses power gating, which is a key technique to reduce power in sleep mode. Because subthreshold leakage is usually the dominant source of static power, other techniques for leakage reduction are explored, including multiple threshold voltages, variable threshold voltages, and stack forcing.

5.3.1 Static Power Sources

As given in EQ (5.12), static power arises from subthreshold, gate, and junction leakage currents and contention current. Entire books have been written about leakage [Narendra06], but this section summarizes the key effects.

5.3.1.1 Subthreshold Leakage Subthreshold leakage current flows when a transistor is supposed to be OFF. It is given by EQ (2.45). For V_{ds} exceeding a few multiples of the thermal voltage (e.g., $V_{ds} > 50$ mV), it can be simplified to

$$I_{sub} = I_{off} 10^{\frac{V_{gs} + \eta(V_{ds} - V_{DD}) - k_\gamma V_{sb}}{S}} \tag{5.19}$$

where I_{off} is the subthreshold current at $V_{gs} = 0$ and $V_{ds} = V_{DD}$, and S is the subthreshold slope given by EQ (2.44) (about 100 mV/decade). I_{off} is a key process parameter defining the leakage of a single OFF transistor. It ranges from about 100 nA/μm for typical low-V_t devices to below 1 nA/μm for high-V_t devices. η is the DIBL coefficient, typically around 100 mV/V for a 65 nm transistor, and trending upward because the drain exerts an increasing influence on the channel as the geometry shrinks. If V_{ds} is small, I_{sub} may decrease by roughly an order of magnitude from I_{off}. k_γ is the body effect coefficient, which describes how the body effect modulates the threshold voltage. Raising the source voltage or applying a negative body voltage can further decrease leakage.

I_{off} is usually specified at 25 °C and increases exponentially with temperature because V_t decreases with temperature and S is directly proportional to temperature. I_{off} typically increases by one to two orders of magnitude at 125 °C, so limiting die temperature is essential to controlling leakage.

The leakage through two or more series transistors is dramatically reduced on account of the *stack effect* [Ye98, Narendra01]. Figure 5.20 shows two series OFF transistors with gates at 0 volts. The drain of $N2$ is at V_{DD}, so the stack will leak. However, the middle node voltage V_x settles to a point that each transistor has the same current. If V_x is small, $N1$ will see a much smaller DIBL effect and will leak less. As V_x rises, V_{gs} for $N2$ becomes negative, reducing its leakage. Hence, we would expect that the series transistors leak less. This can be demonstrated mathematically by solving for V_x and I_{sub}, assuming that $V_x > 50$ mV.

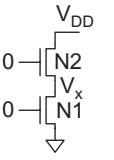


FIGURE 5.20
Series OFF transistors demonstrating the stack effect

$$I_{sub} = \underbrace{I_{off}10^{\frac{\eta(V_x - V_{DD})}{S}}}_{N2} = \underbrace{I_{off}10^{\frac{-V_x + \eta\left((V_{DD} - V_x) - V_{DD}\right) - k_\gamma V_x}{S}}}_{N1} \tag{5.20}$$

$$V_x = \frac{\eta V_{DD}}{1 + 2\eta + k_\gamma} \tag{5.21}$$

$$I_{sub} = I_{off}10^{\frac{-\eta V_{DD}\left(\frac{1+\eta+k_\gamma}{1+2\eta+k_\gamma}\right)}{S}} \approx I_{off}10^{\frac{-\eta V_{DD}}{S}} \tag{5.22}$$

Using the typical values above and $V_{DD} = 1.0$ V, we find that the stack effect reduces subthreshold leakage by a factor of about 10. Stacks with three or more OFF transistors have even lower leakage.

Subthreshold leakage cannot be reduced without consideration of other forms of leakage [Mukhopadhyay05]. Raising the halo doping level to raise V_t by controlling DIBL and short-channel effects causes BTBT to increase. Applying a reverse body bias to increase V_t also causes BTBT to increase. Applying a negative gate voltage to turn the transistor OFF more strongly causes GIDL to increase. Figure 5.21 shows how subthreshold leakage dominates in a 50 nm process at low V_t, but how the other sources take over at higher V_t [Agarwal07].

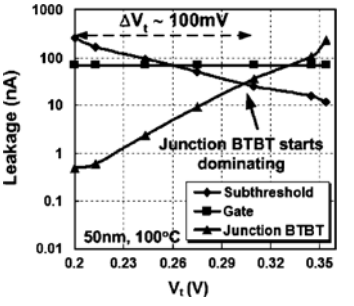


FIGURE 5.21
Leakage as a function of V_t (© IEEE 2007.)

Silicon on Insulator (SOI) circuits are attractive for low-leakage designs because they have a sharper subthreshold current rolloff (smaller n in EQ (2.42)). SOI circuit design will be discussed further in Section 9.5.

5.3.1.2 Gate Leakage Gate leakage occurs when carriers tunnel through a thin gate dielectric when a voltage is applied across the gate (e.g., when the gate is ON). A process usually specifies I_G in nA/μm for a minimum-length gate or in A/mm^2 of transistor gate. Gate leakage is an extremely strong function of the dielectric thickness. It is normally limited to acceptable levels in the process by selection of the dielectric thickness. pMOS gate leakage is an order of magnitude smaller in ordinary SiO_2 gates and can often be ignored, but it can be significant for other gate dielectrics.

Gate leakage also depends on the voltage across the gate. For example, Figure 5.22 shows two series transistors. If $N1$ is ON and $N2$ is OFF, $N1$ has $V_{gs} = V_{DD}$ and experiences full gate leakage. On the other hand, if $N1$ is OFF and $N2$ is on, $N2$ has $V_{gs} = V_t$ and

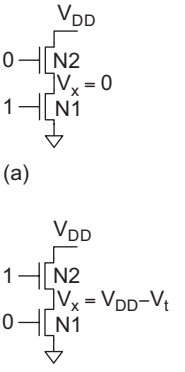


FIGURE 5.22
Gate leakage in series stack

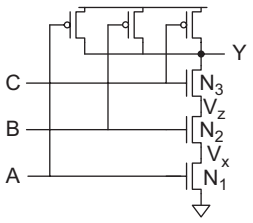


FIGURE 5.23 NAND gate demonstrating pattern dependence of gate and subthreshold leakage

experiences negligible gate leakage [Lee03, Mukhopadhyay03]. In both cases, the OFF transistor has no gate leakage. Thus, gate leakage can be alleviated by stacking transistors such that the OFF transistor is closer to the rail.

Table 5.2 summarizes the combined effects of gate and subthreshold leakage on the 3-input NAND gate shown in Figure 5.23 using data from [Lee03] for a process with 15 Å oxides and 60 nm channel length. The gate leakage through an ON nMOS transistor is 6.3 nA. pMOS gate leakage is negligible. The subthreshold leakage through an nMOS transistor with $V_{ds} = V_{DD}$ is 5.63 nA and the subthreshold leakage through a pMOS transistor with $|V_{ds}| = V_{DD}$ is 9.3 nA.

The NAND3 benefits from the stack effect to reduce subthreshold leakage. In the 000 case, all three nMOS transistors are OFF and the triple stack effect cuts leakage by a factor of 10. Both intermediate nodes drift up to somewhere around 100–200 mV set by the stack effect. In the 001 and 100 cases, two nMOS transistors are OFF and the double stack effect cuts leakage by a factor of 5. In the 110 case, the nMOS stack experiences full subthreshold leakage because only one transistor is OFF and it sees $V_{ds} = V_{DD}$. In the 011 and 101 cases, the single OFF nMOS transistor sees $V_{ds} = V_{DD} - V_t$, so the leakage is partially reduced. In the 111 case, all three parallel pMOS transistors leak.

The NAND3 also sees pattern-dependent gate leakage. In the 000 case, all three nMOS transistors are off, so no gate current flows. In the 001 and 011 cases, the ON transistors see $V_{gs} = V_t$ and thus have little leakage. In the 010 case, gate leakage through $N2$ charges V_x and V_z up to an intermediate voltage until the increase in source/drain voltage reduces the gate current. This raises the source voltage of $N3$, effectively eliminating its subthreshold leakage. In the 101 case, $N1$ sees full gate leakage, while $N3$ has little because V_z is at a high voltage. In the 110 case, $N1$ and $N2$ both see gate leakage, and in the 111 case, all three nMOS transistors leak.

TABLE 5.2 Gate and subthreshold leakage in NAND3 (nA)

Input State (ABC)	I_{sub}	I_{gate}	I_{total}	V_x	V_z
000	0.4	0	0.4	stack effect	stack effect
001	0.7	0	0.7	stack effect	$V_{DD} - V_t$
010	0	1.3	1.3	intermediate	intermediate
011	3.8	0	10.1	$V_{DD} - V_t$	$V_{DD} - V_t$
100	0.7	6.3	7.0	0	stack effect
101	3.8	6.3	10.1	0	$V_{DD} - V_t$
110	5.6	12.6	18.2	0	0
111	28	18.9	46.9	0	0

5.3.1.3 Junction Leakage Junction leakage occurs when a source or drain diffusion region is at a different potential from the substrate. Although the ordinary leakage of reverse-biased diodes is usually negligible, BTBT and GIDL can result in leakage currents that approach subthreshold leakage levels in high-V_t transistors. BTBT is maximum when a strong reverse bias is applied between the drain and body (e.g., $V_{db} = V_{DD}$ for an nMOS transistor). GIDL is maximum when the transistor is OFF and a strong bias is applied to the drain (e.g., $V_{gd} = -V_{DD}$ for an nMOS transistor). Junction leakage is often

minor in comparison to the other leakages, but can be expressed in nA/μm of transistor width when it needs to be considered.

5.3.1.4 Contention Current Static CMOS circuits have no contention current. However, certain alternative circuits inherently draw current even while quiescent. For example, pseudo-nMOS gates discussed in Section 9.2.2 experience contention between the nMOS pulldowns and the always-on pMOS pullups when the output is 0. Current-mode logic and many analog circuits also draw static current. Such circuits should be turned OFF in sleep mode by disabling the pullups or current source.

5.3.1.5 Static Power Estimation Static current estimation is a matter of estimating the total width of transistors that are leaking, multiplying by the leakage current per width, and multiplying by the fraction of transistors that are in their leaky state (usually one-half). Add the contention current if applicable. The static power is the supply voltage times the static current.

Example 5.4

Consider the system-on-chip from Example 5.1. Subthreshold leakage for OFF devices is 100 nA/μm for low-threshold devices and 10 nA/μm for high-threshold devices. Gate leakage is 5 nA/μm. Junction leakage is negligible. Memories use low-leakage devices everywhere. Logic uses low-leakage devices in all but 5% of the paths that are most critical for performance. Estimate the static power consumption.

SOLUTION: There are $(50 \times 10^6$ logic transistors$)(0.05)(12\ \lambda)(0.025\ \mu m/\lambda) = 0.75 \times 10^6$ μm of low-threshold devices and $[(50 \times 10^6$ logic transistors$)(0.95)(12\ \lambda) + (950 \times 10^6$ memory transistors$)(4\ \lambda)](0.025\ \mu m/\lambda) = 109.25 \times 10^6$ μm of high-threshold devices. Neglecting the benefits of series stacks, half the transistors are OFF and contribute subthreshold leakage. Half the transistors are ON and contribute gate leakage. $I_{sub} = [(0.75 \times 10^6\ \mu m)(100\ nA/\mu m) + (109.25 \times 10^6\ \mu m)(10\ nA/\mu m)]/2 = 584$ mA. $I_{gate} = ((0.75 + 109.25) \times 10^6\ \mu m)(5\ nA/\mu m)/2 = 275$ mA. $P_{static} = (584\ mA + 275\ mA)(1\ V) = 859$ mW. This is 15% of the switching power and is enough to deplete the battery of a hand-held device rapidly.

5.3.2 Power Gating

The easiest way to reduce static current during sleep mode is to turn off the power supply to the sleeping blocks. This technique is called *power gating* and is shown in Figure 5.24. The logic block receives its power from a virtual V_{DD} rail, V_{DDV}. When the block is active, the header switch transistors are ON, connecting V_{DDV} to V_{DD}. When the block goes to sleep, the header switch turns OFF, allowing V_{DDV} to float and gradually sink toward 0. As this occurs, the outputs of the block may take on voltage levels in the forbidden zone. The output isolation gates force the outputs to a valid level during sleep so that they do not cause problems in downstream logic.

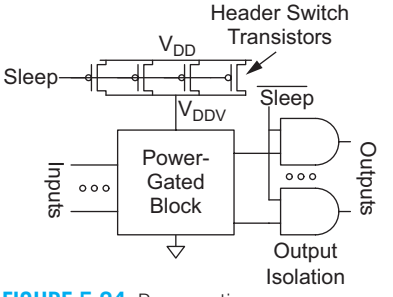


FIGURE 5.24 Power gating

Power gating introduces a number of design issues. The header switch requires careful sizing. It should add minimal delay to the circuit during active operation, and should have low leakage during sleep. The transition between active and sleep modes takes some time and energy, so power gating is only effective when a block is turned off long enough. When a block is gated, the

state must either be saved or reset upon power-up. Section 10.4.3 discusses state retention registers that use a second power supply to maintain the state. Alternatively, the important registers can be saved to memory so the entire block can be power-gated. The registers must then be reloaded from memory when power is restored. [Keating07] addresses at length how to use power gating in a standard CAD flow. If power switches are fast enough, they can be used to save leakage power during active mode by powering down clock-gated blocks [Tschanz03, Min06]. If saving or losing the state costs too much overhead, turning the power supply down to the minimum level necessary to retain state (about 300 mV) using DVS is sufficient to eliminate gate leakage and reduce subthreshold leakage energy by more than an order of magnitude [Calhoun04].

Power gating was originally proposed as *Multiple Threshold CMOS* (MTCMOS) [Mutoh95] because it used low-V_t transistors for logic and high-V_t header and footer switches. However, the name is somewhat confusing because a system may use multiple threshold voltages without power gating. Moreover, it is unnecessary to switch both V_{DD} and GND.

5.3.2.1 Power Gate Design Power gating can be done externally with a disable input to a voltage regulator or internally with high-V_t header or footer switches. External power gating completely eliminates leakage during sleep, but it takes a long time and significant energy because the power network may have 100s of nF of decoupling capacitance to discharge.

On-chip power gating can use pMOS header switch transistors or nMOS footer switch transistors. nMOS transistors deliver more current per unit width so they can be smaller. On the other hand, if both internal and external power gating are used, it is more consistent for both methods to cut off V_{DD}. pMOS power gating also is simpler when multiple power supplies are employed. As a practical matter, ensuring that GND is always constant reduces confusion among designers and CAD tools; this alone is enough for many projects to choose pMOS power gating.

Theoretically, it is possible to use *fine-grained power gating* applied to individual logic gates, but placing a switch in every cell has enormous area overhead. Practical designs use *coarse-grained power gating* where the switch is shared across an entire block. The switch has an effective resistance that inevitably causes some voltage droop on V_{DDV} and increases the delay of the block. The switch is commonly sized to keep this delay to 5–10%. One way to achieve this is to calculate or simulate how much voltage droop can occur on V_{DDV} while maintaining acceptable delay. Then the average current of the block is determined through power analysis. The switch width is chosen so that the voltage droop is small enough when the average current flows through the switch. If the block is large enough that switching events are spread over time and has enough capacitance on V_{DDV} to smooth out ripples, this *average current method* [Mutoh99] is satisfactory. Wider switches reduce the droop but have more leakage when OFF and take more energy. For example, 45 nm Core processors use 1.5 meters of low-leakage pMOS power gate transistor per core to turn off the idle cores [Kumar09].

Example 5.5

A cache in a 65 nm process consumes an average power of 2 W. Estimate how wide should the pMOS header switch be if delay should not increase by more than 5%?

SOLUTION: The 65 nm process operates at 1 V, so the average current is 2 W / 1 V = 2 A. The pMOS transistor has an ON resistance of $R = 2\ \text{k}\Omega \cdot \mu\text{m}$. A 5% delay increase

corresponds to a droop on V_{DDV} of about 5% (check this using EQ (4.29). Thus, $R_{switch} = 0.05 \times 1\ V / 2\ A = 25\ m\Omega$. So the transistor width must be $k\Omega \cdot \mu m / 25\ m\Omega = 8 \times 10^4\ \mu m$. The ON resistance at low V_{ds} is lower than R. Circuit simulation shows that a width of $3.7 \times 10^4\ \mu m$ suffices to keep droop to 5%.

The power switch is generally made of many transistors in parallel. The length and width of the transistors should be selected to maximize the I_{on} / I_{off} ratio; this is highly process-dependent and generally requires SPICE simulations sweeping L and W. A reverse body bias may be applied to the power switch transistors during sleep mode to improve their I_{on} / I_{off} ratio (see Section 5.3.4). Alternatively, the switch can be overdriven positively or negatively to turn it ON or OFF more effectively so long as the gate oxide is not overstressed [Min06].

When the power switch is turned ON, the sudden inrush of current can cause *IR* and *L di/dt* drop noise (see Section 13.3) and electromigration of the power bus (see Section 7.3.3.1). To alleviate these problems, the switch can be turned on gradually by controlling how many parallel transistors are ON.

5.3.3 Multiple Threshold Voltages and Oxide Thicknesses

Selective application of multiple threshold voltages can maintain performance on critical paths with low-V_t transistors while reducing leakage on other paths with high-V_t transistors.

A multiple-threshold cell library should contain cells that are physically identical save for their thresholds, facilitating easy swapping of thresholds. Good design practice starts with high-V_t devices everywhere and selectively introduces low-V_t devices where necessary.

Using multiple thresholds requires additional implant masks that add to the cost of a CMOS process. Alternatively, designers can increase the channel length, which tends to raise the threshold voltage via the short channel effect. For example, in Intel's 65 nm process, drawing transistors 10% longer reduces I_{on} by 10% but reduces I_{off} by a factor of 3 [Rusu07]. The dual-core Xeon processor uses longer transistors almost exclusively in the caches and in 54% of the core gates.

Most nanometer processes offer a thin oxide for logic transistors and a much thicker oxide for I/O transistors that can withstand higher voltages. The oxide thickness is controlled by another mask step. Gate leakage is negligible in the thick oxide devices, but their performance is inadequate for high speed logic applications. Some processes offer another intermediate oxide thickness to reduce gate leakage.

[Anis03] provides an extensive survey of the applications of multiple thresholds.

5.3.4 Variable Threshold Voltages

Recall from EQ (2.38) that V_{sb} modulates the threshold voltage through the body effect. Another method to achieve high I_{on} in active mode and low I_{off} in sleep mode is to dynamically adjust the threshold voltage of the transistor by applying a body bias. This technique is sometimes called *variable threshold CMOS* (VTCMOS).

For example, low-V_t devices can be used and a *reverse body bias* (RBB) can be applied during sleep mode to reduce leakage [Kuroda96]. Alternatively, higher-V_t devices can be used, and then a *forward body bias* (FBB) can be applied during active mode to increase performance [Narendra03]. Body bias can be applied to the power gating transistors to turn them off more effectively during sleep.

Too much reverse body bias (e.g., < −1.2 V) leads to greater junction leakage through BTBT [Keshavarzi01], while too much forward body bias (> 0.4 V) leads to substantial current through the body to source diodes. According to EQ (2.39), the body effect weakens as t_{ox} becomes thinner, so body biasing offers diminishing returns at 90 nm and below [von Arnim05].

Applying a body bias requires additional power supply rails to distribute the substrate and well voltages. For example, an RBB scheme for a 1.0 V n-well process could bias the p-type substrate at $V_{BBn} = -0.4$ V and the n-well at $V_{BBp} = 1.4$ V. Figure 5.25 shows a schematic and cross-section of an inverter using body bias. In an n-well process, all nMOS transistors share the same p substrate and must use the same V_{BBn}. In a triple-well process, groups of transistors can use different p-wells isolated from the substrate and thus can use different body biases. The well and substrate carry little current, so the bias voltages are relatively easy to generate using a charge pump (see Section 13.3.8).

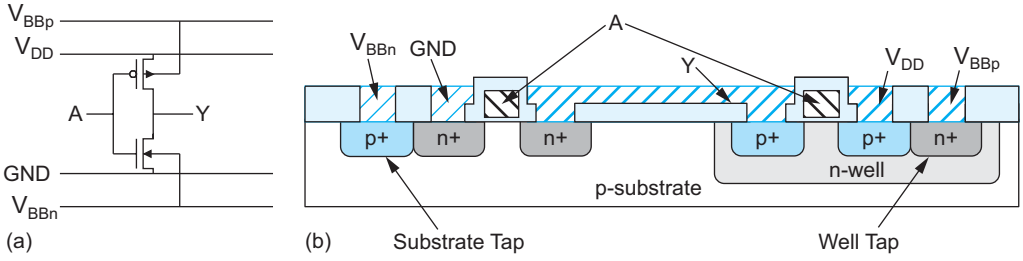


FIGURE 5.25 Body bias

5.3.5 Input Vector Control

As was illustrated in Table 5.2, the stack effect and input ordering cause subthreshold and gate leakage to vary by up to two orders of magnitude between best and worst cases. Therefore, the leakage of a block of logic depends on gate inputs, which in turn depend on the inputs to the block of logic. The idea of *input vector control* is to apply the input pattern that minimizes block leakage when the block is placed in sleep mode [Narendra06, Abdollahi04]. The vector can be applied via set/reset inputs on the registers or via a scan chain. It is hard to control all the gates in a block of logic using only the block inputs, but the best input vectors may save 25–50% of leakage as compared to random vectors. Applying the input vector causes some switching activity, so a block may need to remain in sleep for thousands of cycles to recoup the energy spent entering the sleep state.

5.4 Energy-Delay Optimization

At this point, a natural question is: what is the best choice of V_{DD} and V_t? The answer, of course, depends on the objective. Minimum power by itself is not an interesting objective because it is achieved as the delay for a computation approaches infinity and nothing is accomplished. The time for a computation must be factored into the analysis. Better metrics include minimizing the energy, minimizing the energy-delay product, and minimizing energy under a delay constraint.

5.4.1 Minimum Energy

According to EQ (5.3), the product of the power of an operation and the time for the operation to complete is the energy consumed. Hence, the *power-delay product* (PDP) is simply the

energy. The minimum energy point is the least energy that an operation could consume if delay were unimportant. It occurs in subthreshold operation where $V_{DD} < V_t$. The minimum energy point typically consumes an order of magnitude less energy than the conventional operating point, but runs at least three orders of magnitude more slowly [Wang06].

John von Neumann first asserted (without justification) that the "thermodynamic minimum of energy per elementary act of information" was $kT \ln 2$ [von Neumann66]. [Meindl00] proved this result for CMOS by considering the minimum allowable voltage at which an inverter could operate. To achieve nonzero noise margins, an inverter must have a slope steeper than −1 at the switching point, V_{inv}. For an ideal inverter with $n = 1$ in the subthreshold characteristics, this occurs at a minimum operating voltage of

$$V_{min} = 2 \ln 2 v_T = 36 \text{ mV} @ 300 \text{ K} \quad \textbf{(5.23)}$$

The energy stored on the gate capacitance of a single MOSFET is $E = QV_{DD}/2$, where Q is the charge. The minimum possible charge is one electron, q. Substituting V_{min} for V_{DD} gives $E_{min} = kT \ln 2 = 2.9 \times 10^{-21}$ J. In contrast, a unit inverter in a 0.5 μm 5 V process draws about 1.5×10^{-13} J from the supply when switching, and the same inverter in a 65 nm 1 V process draws 3×10^{-16} J.

Inverters have been demonstrated operating with power supplies under 100 mV, but these do not actually minimize energy in a real CMOS process. Although they have extremely low switching energy, they run so slowly that the leakage energy dominates. The true minimum energy point is at a higher voltage that balances switching and leakage energy.

In subthreshold operation, the current drops exponentially as $V_{DD} - V_t$ decreases and thus the delay increases exponentially. The switching energy improves quadratically with V_{DD}. Leakage current improves slowly with V_{DD} because of DIBL, but the leakage energy increases exponentially because the slower gate leaks for a longer time. To achieve minimum energy operation, all transistors should be minimum width. This reduces both switching capacitance and leakage. Gate and junction leakage and short-circuit power are negligible in subthreshold operation, so the total energy is the sum of the switching and leakage energy, which is minimized near the point they crossover, as shown in Figure 5.26.

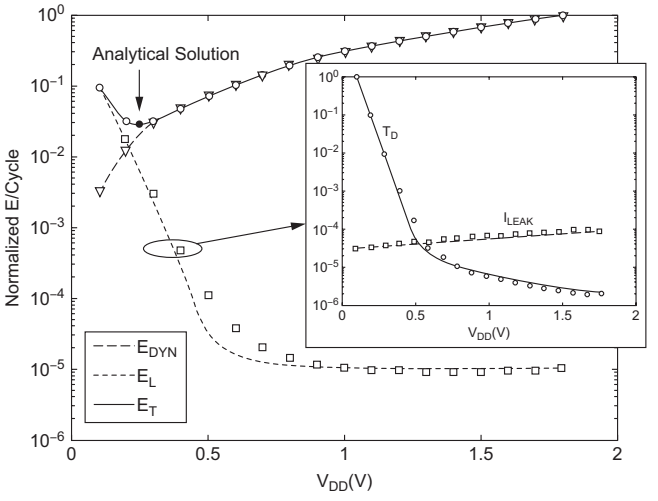


FIGURE 5.26 Minimum energy point (Reprinted from [Calhoun05]. © 2005 IEEE.)

To compute the energy, assume that a circuit has N gates on the critical path, a total effective capacitance C_{eff}, and a total effective width W_{eff} of leaking transistor. The delay of a gate operating subthreshold with a load C_g is given by EQ (4.31). The cycle time is thus

$$D = \frac{NkC_gV_{DD}}{I_{\text{off}}10^{V_{DD}}} \tag{5.24}$$

The energy consumed in one cycle is

$$\begin{aligned} E_{\text{switching}} &= C_{\text{eff}}V_{DD}^2 \\ E_{\text{leak}} &= I_{\text{sub}}V_{DD}D = W_{\text{eff}}NkC_g10^{-V_{DD}}V_{DD}^2 \\ E_{\text{total}} &= E_{\text{switching}} + E_{\text{leak}} = V_{DD}^2\left(C_{\text{eff}} + W_{\text{eff}}NkC_g10^{-V_{DD}}\right) \end{aligned} \tag{5.25}$$

It is possible to differentiate EQ (5.25) with respect to V_{DD} to find the minimum energy point [Calhoun05], but the results are rather messy.

A more intuitive approach is to look at the minimum energy point graphically. Figure 5.27(a) plots the energy and delay contours as a function of V_{DD} and V_t for a ring oscillator in a 180 nm process designed to reflect the behavior of a microprocessor pipeline [Wang02]. As V_{DD} increases or V_t decreases, the operating frequency increases exponentially assuming the circuit is operating at or near threshold. At $V_{DD} = V_t$, the circuit operates at about 10 MHz. The energy contours are normalized to the minimum energy point. This point, marked with a cross, occurs at $V_{DD} = 0.13$ V and $V_t = 0.37$ V. The energy is about 10 times lower than at a typical operating point, but the delay is three to four orders of magnitude greater.

The shape of the curve is only a weak function of process parameters, so it remains valid for nanometer processes. However, the result does depend strongly on the relative switching and leakage energies. Figure 5.27(b) plots the results when the activity factor drops to 0.1, reducing C_{eff}. Switching energy is less important, so the circuit can run at a higher supply voltage. The threshold then increases to cut leakage. The total energy is

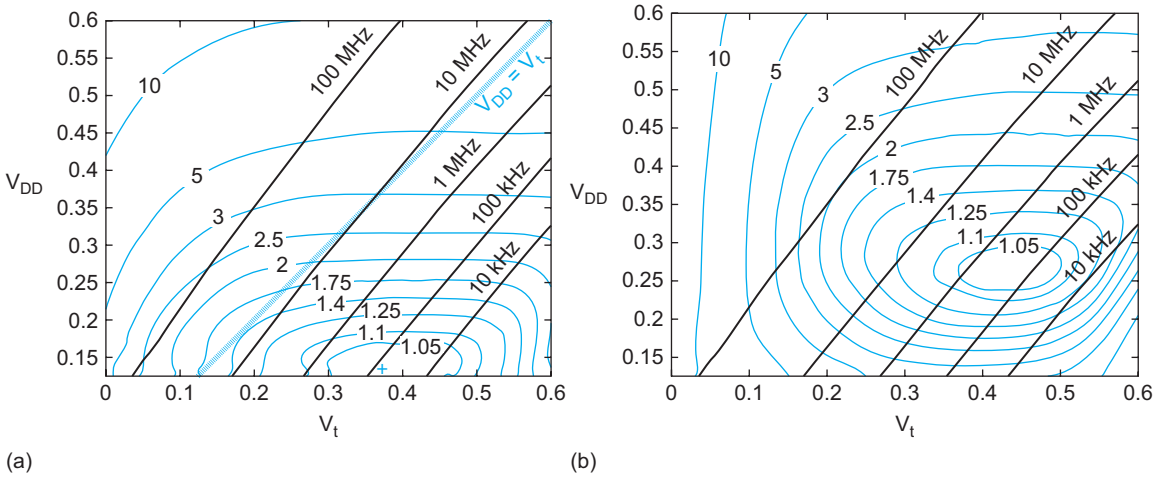


FIGURE 5.27 Contours of energy and delay for ring oscillators with (a) $\alpha = 1$, (b) $\alpha = 0.1$ (Adapted from [Wang02]. © IEEE 2002.)

greatly reduced. The result also depends on temperature: at high temperature, circuits leak more so a higher threshold voltage should be used. Process variation also pushes the best operating point toward higher voltage and energy.

5.4.2 Minimum Energy-Delay Product

The *energy-delay product* (EDP) is a popular metric that balances the importance of energy and delay [Gonzalez97, Stan99, Nose00c]. Neglecting leakage, we can elegantly solve for the supply voltage that minimizes EDP. Considering leakage, the best supply voltage is slightly higher.

First, consider the EDP when leakage is negligible. The energy to charge a load capacitance C_{eff} is given by EQ (5.7). The delay, using an α-power law model, is given by EQ (4.29). Thus, the EDP is

$$\text{EDP} = k\frac{C_{\text{eff}}^2 V_{DD}^3}{\left(V_{DD} - V_t\right)^{\alpha}} \tag{5.26}$$

Differentiating with respect to V_{DD} and setting the result to 0 gives the voltage at which the EDP is minimized

$$V_{DD\text{-opt}} = \frac{3}{3-\alpha}V_t \tag{5.27}$$

Recall that α is between 1 (completely velocity saturated) and 2 (no velocity saturation). For a typical value of α, we come to the interesting conclusion that $V_{DD\text{-opt}} \approx 2V_t$, which is substantially lower than most systems presently run.

EQ (5.26) suggests that the EDP improves as V_t approaches 0, which is obviously not true because leakage power would dominate. When a leakage term is incorporated into EQ (5.27), the results become too messy to reprint here. Figure 5.28 shows contours of EDP and delay as a function of V_{DD} and V_t. EDP is normalized to the best achievable. For typical process parameters, the best V_t is about 100–150 mV and the EDP is about four times better than at a typical operating point of $V_{DD} = 1.0$ V and $V_t = 0.3$ V. At the optimum, leakage energy is about half of dynamic energy. The dashed lines indicate contours of equal speed, normalized to the speed at the best EDP point. To operate at higher speed requires increasing the EDP. Section 7.5.3.2 will revisit this analysis considering process variation and show that the minimum EDP point occurs at a higher voltage and threshold when variations are accounted for.

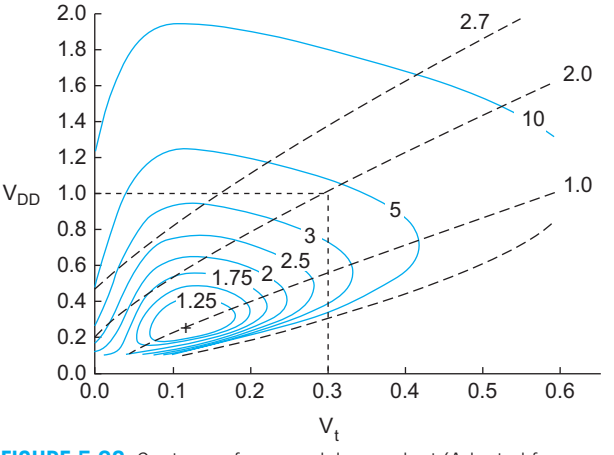


FIGURE 5.28 Contours of energy-delay product (Adapted from [Gonzalez97]. © IEEE 1997.)

5.4.3 Minimum Energy Under a Delay Constraint

In practice, designers generally face the problem of achieving minimum energy under a delay constraint. Equivalently, the power consumption of the system is limited by battery or cooling considerations and the designer seeks to achieve minimum delay under an

energy constraint. Figure 5.27(a) showed contours of delay and energy. The best supply voltage and threshold for operation at a given delay is where the delay and energy contours are tangent.

For a given supply voltage and threshold voltage, the designer can make logic and sizing choices that affect delay and energy. Figure 5.13 showed an example of an energy-delay trade-off curve. Such curves can be generated using a logic synthesizer or sizing tool constrained to various delays. The curve becomes steep near the point of minimum delay, so energy-efficient designs should aim to operate at a longer delay.

Energy under a delay constraint is also minimized when leakage is about half of dynamic power [Marković04]. However, the curve is fairly flat around this point, so many designs operate at lower leakage to facilitate power saving during sleep mode.

5.5 Low Power Architectures

VLSI design used to be constrained by the number of transistors that could fit on a chip. Extracting maximum speed from each transistor maximized overall performance. Now that billions of nanometer-scale transistors fit on a chip, many designs have become power constrained and the most energy-efficient design is the highest performer. This is one of the factors that has driven the industry's abrupt shift to multicore processors.

5.5.1 Microarchitecture

Energy-efficient architectures take advantage of the structured design principles of modularity and locality [Horowitz04, Naffziger0b]. [Pollack99] observed that processor performance grows with the square root of the number of transistors. Building complex, sprawling processors to extract the last bit of instruction-level parallelism from a problem is a highly inefficient use of energy. Microarchitectures are moving toward larger numbers of simpler cores seeking to handle task and data-level parallelism. Smaller cores also have shorter wires and faster memory access.

Memories have a much lower power density than logic because their activity factors are miniscule and their regularity simplifies leakage control. If a task can be accelerated using either a faster processor or a larger memory, the memory is often preferable. Memories now comprise more than half the area of many chips.

Special-purpose functional units can offer an order of magnitude better energy efficiency than general-purpose processors. Accelerators for compute-intensive applications such as graphics, networking, and cryptography offload these tasks from the processor. Such heterogeneous architectures, combining regular cores, specialized accelerators, and large amounts of memory, are of growing importance.

Commercial software has historically lagged at least a decade behind hardware advances such as virtual memory, memory protection, 32- and 64-bit datapaths, and robust power-management. Presently, programmers have trouble taking advantage of many cores. Time will tell whether programming practices and tools catch up or whether microarchitectures will have to yield to the needs of programmers.

5.5.2 Parallelism and Pipelining

In the past, *parallelism* and *pipelining* have been effective ways to reduce power consumption, as shown in Figure 5.29 [Chandrakasan92].

Replacing a single functional unit with N parallel units allows each to operate at $1/N$ the frequency. A multiplexer selects between the results. The voltage can be scaled down accordingly, offering quadratic savings in energy at the expense of doubling the area. Replacing a single functional unit with an N-stage pipelined unit also reduces the amount of logic in a clock cycle at the expense of more registers. Again, the voltage can be scaled down. The two techniques can be combined for even better energy efficiency.

When leakage is unimportant, parallelism offers a slight edge because the multiplexer has less overhead than the pipeline registers. Also, perfectly balancing logic across pipeline stages can be difficult. Now that leakage is a substantial fraction of total power, pipelining becomes preferable because the parallel hardware has N times as much leakage [Markovi\'c04].

Now that V_{DD} is closer to the best energy-delay point, the potential supply reduction and energy savings are diminishing. Nevertheless, parallelism and pipelining remain primary tools to extract performance from the vast transistor budgets now available.

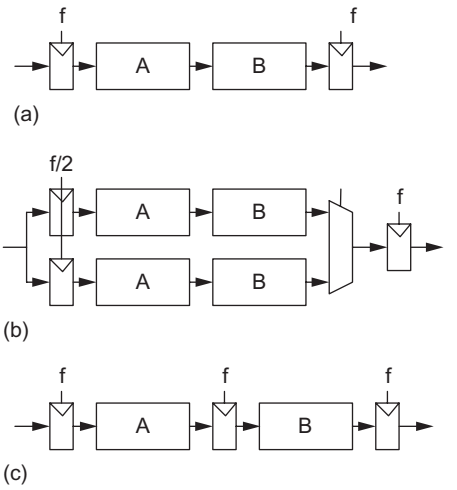


FIGURE 5.29 Functional units: (a) normal, (b) parallel, (c) pipelined

5.5.3 Power Management Modes

As your parents taught you to turn off the lights when you leave a room, chip designers have now learned they must turn off portions of the chip when they are not active by applying clock and power gating. Many chips now employ a variety of power management modes giving a trade-off between power savings and wake-up time.

For example, the Intel Atom processor [Gerosa09] operates at a peak frequency of 2 GHz at 1 V, consuming 2 W. The power management modes are shown in Figure 5.30. In the low frequency mode, the clock drops as slow as 600 MHz while the power supply

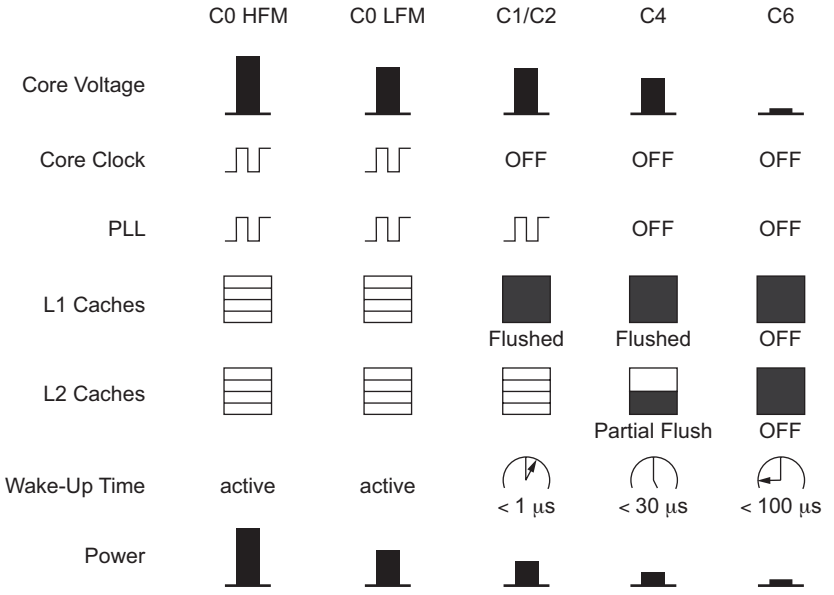


FIGURE 5.30 Atom power management modes (© 2009 IEEE.)

reduces to 0.75 V. In sleep mode C1, the core clock is turned off and the level 1 cache is flushed and power-gated to reduce leakage, but the processor can return to active state in 1 microsecond. In sleep mode C4, the PLL is also turned OFF. In sleep mode C6, the core and caches are all power-gated to reduce power to less than 80 mW, but wake-up time rises to 100 microseconds. For a typical workload, the processor can spend 80–90% of its time in C6 sleep mode, reducing average power to 220 mW.

The worst-case power that a chip may consume can be a factor of two or more greater than the normal power. Code triggering maximal power consumption is sometimes called a *thermal virus* [Naffziger06] because it seeks to burn out the chip. To avoid having to design for this worst case, chips can employ adaptive features, throttling back activity if the issue rate or die temperature becomes too high. Section 13.2.5 discusses temperature sensors further.

Power management results in substantially lower power consumption during idle mode than active mode. The transition between idle and active may require multiple cycles to avoid sudden current spikes that excite power supply resonances and cause excessive supply noise.

5.6 Pitfalls and Fallacies

Oversizing gates

Designers seeking timing closure tend to crank up the size of gates. Doubling the size of all the gates on a gate-dominated path does not improve delay, but doubles the power consumption.

Designing for speed without regard to power

Nanometer processes have reached a point where it is no longer possible to design a large chip for speed without regard to power: the chip will be impossible to cool. Designs must be power efficient. Systems tuned exclusively for speed tend to use large gates and speculative logic that consumes a great deal of power. If a core or processing element can be simplified to offer 80% of the performance at 50% of the power, then two cores in parallel can offer 160% of the throughput at the same power.

Reporting power at a given frequency instead of energy per operation

Sometimes a module is described by its power at an arbitrary frequency (e.g., 10 mW @ 1 GHz). This is equivalent to reporting energy because $E = P/f$ (e.g., 10 pJ). Reporting energy is arguably cleaner because it is a single number.

Reporting Power-Delay Product when Energy-Delay Product is meant

Extending the previous point, sometimes a system is described by its PDP at a given frequency, where the frequency is slower than the reciprocal of the delay. This metric is really a variation of the EDP, because the power at a low enough frequency is equivalent to energy. Reporting the EDP is definitely cleaner because it doesn't involve an arbitrary choice of frequency.

Failing to account for leakage

Many designers are accustomed to focusing on dynamic power. Leakage in all its forms has become extremely important in nanometer processes. Ignoring it not only underestimates power consumption but also can cause functional failures in sensitive circuits.

5.7 Historical Perspective

The history of electronics has been a relentless quest to reduce power so that more capabilities can be provided in a smaller volume.

The Colossus, brought online in 1944, was one of the world's first fully electronic computers. The secret machine was built from 2400 vacuum tubes and consumed 15 kW as it worked day and night decrypting secret German communications. The machine was destroyed after the war, but a functional replica shown in Figure 5.31 was rebuilt in 2007.

Vacuum tube machines filled entire rooms and failed frequently. Imagine the problem of keeping 2400 light bulbs burning simultaneously. By the 1960s, vacuum tubes were surpassed by solid-state transistors that were far smaller and consumed milliwatts rather than watts. Gordon Moore soon issued his famous prophecy about the exponential growth in the number of transistors per chip.

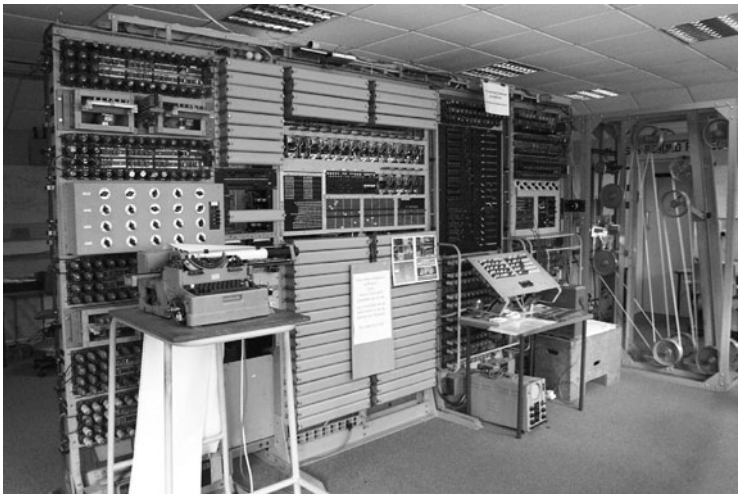


FIGURE 5.31 Reconstructed Colossus Mark 2 (Photograph by Tony Sale. Reprinted with permission.)

MOSFETs entered the scene commercially around 1970. For more than a decade, nMOS technology predominated because it could pack transistors more densely (and hence cheaply) than CMOS. nMOS circuits used *depletion load* (negative-V_t) nMOS pullups as resistive loads, so each gate with an output of 0 dissipated contention power. For example, Figure 5.32 shows an nMOS 2-input NOR gate.

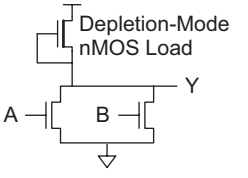


FIGURE 5.32 nMOS NOR gate

CMOS circuits made their debut in watch circuits (pioneered by none other than the Swiss, of course!), where their key ability to draw almost zero power while not switching was critical [Vittoz72]. This use succeeded despite very low circuit densities and low circuit speed of the CMOS technologies of the day. It was not until the mid 1980s that the ever-increasing power dissipation of mainstream circuits such as microprocessors forced a move from nMOS to CMOS technology, again despite density arguments.

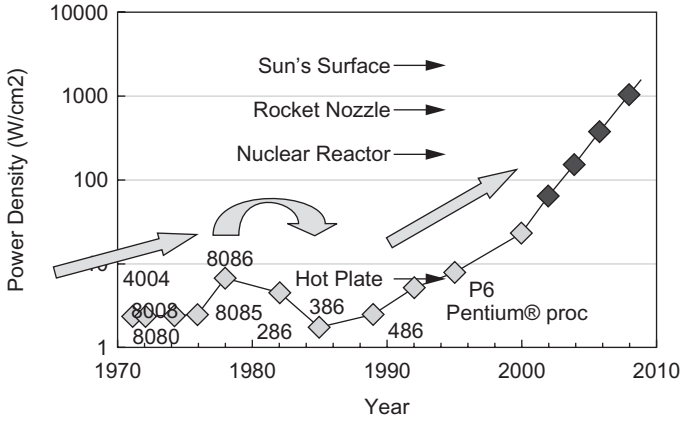


FIGURE 5.33 Microprocessor power density trends as predicted in 2001 (Reprinted with permission of Intel Corporation.)

As engineers became capable of integrating millions (and now billions) of CMOS transistors onto a single chip, power consumption became a concern for CMOS as well. In the 1990s, designers facing the power wall abandoned the long-cherished 5 V standard and began scaling power supplies to reduce dynamic power. Eventually, this forced threshold voltages to decrease until subthreshold leakage has become an issue. As gate dielectrics have scaled down to a few atoms in thickness, quantum mechanical effects have made gate leakage problematic as well. With supplies at 1 V, there is limited room for further power scaling: we are stuck between a rock and a hot place.

On February 5, 2001, Intel Vice President Patrick Gelsinger gave a keynote speech at the International Solid State Circuits Conference [Gelsinger01]. He showed that microprocessor power consumption has been increasing exponentially (see Figure 5.33) and was forecast to grow even faster in the coming decade. He predicted that "business as usual will not work in the future," and that if scaling continued at this pace, by 2005, high-speed processors would have the power density of a nuclear reactor, by 2010, a rocket nozzle, and by 2015, the surface of the sun! Obviously, business did not proceed as usual and power consumption has leveled out at under 150 W for high-performance processors and much lower for battery-powered systems.

Clock gating was the first widely applied technique for power reduction because it is relatively straightforward. Power gating was initially applied to low-power battery-operated systems to increase the standby lifetime, but is now required for leakage control even in high-performance microprocessors [Rusu10]. Voltage domains are also widely used. Initially, separate supplies were provided for the core and I/O to provide compatibility with legacy I/O standards. The next step was to separate the supplies for the memories from the core logic. Memories arrays often use a constant relatively high supply voltage (as high as the process allows) for reliability. Logic dissipates the bulk of the dynamic power, so it operates at a lower, possibly variable voltage. Sometimes phase-locked loops or sensitive analog circuitry use yet another filtered domain. Dynamic voltage scaling is commonly used to support a range of power/performance trade-offs [Clark01]. For example, laptop processors commonly run at a higher voltage when the system is plugged into wall power.

Body bias has been used for leakage control in applications such as the Intel XScale microprocessor [Clark02], the Transmeta Efficeon microprocessor, and a Toshiba MPEG4 video codec [Takahashi98]. Clustered voltage scaling was also used in the video codec. Both of these techniques introduce overhead routing the bias or voltage lines through a block and controlling noise on these lines. They have not achieved the widespread popularity of other techniques, and the effectiveness of body bias becomes limited below 130 nm because the body effect coefficient decreases along with oxide thickness.

The move to CMOS technology was really the last major movement in mass-market semiconductor technologies. To date, no one has come up with better devices. The hundreds of billions of dollars that have been invested in optimizing CMOS make it a formidable technology to surpass. Rather than looking for a replacement, our best hope is to continue learning to use energy as efficiently as we can.

Summary

The power consumption of a circuit has both dynamic and static components. The dynamic power comes from charging and discharging the load capacitances and depends on the frequency, voltage, capacitance, and activity factor. The static power comes from leakage and from circuits that have an intentional path from V_{DD} to GND. CMOS circuits have historically consumed relatively low power because complementary CMOS gates dissipate almost zero static power when operated at high V_t. However, leakage is increasing as feature size decreases, making static power consumption as great a concern as dynamic power. The best way to control power is to turn off a circuit when it is not in use. The most important techniques are clock gating, which turns off the clock when a unit is idle, and power gating, which turns off the power supply when a unit is in sleep mode.

Exercises

5.1 You are synthesizing a chip composed of random logic with an average activity factor of 0.1. You are using a standard cell process with an average switching capacitance of 450 pF/mm^2. Estimate the dynamic power consumption of your chip if it has an area of 70 mm^2 and runs at 450 MHz at $V_{DD} = 0.9$ V.

5.2 You are considering lowering V_{DD} to try to save power in a static CMOS gate. You will also scale V_t proportionally to maintain performance. Will dynamic power consumption go up or down? Will static power consumption go up or down?

5.3 The stack effect causes the current through two series OFF transistors to be an order of magnitude less than I_{off} when DIBL is significant. Show that the current is $I_{off}/2$ when DIBL is insignificant (e.g., $\eta = 0$). Assume $\gamma = 0$, $n = 1$.

5.4 Determine the activity factor for the signal shown in Figure 5.34. The clock rate is 1 GHz.

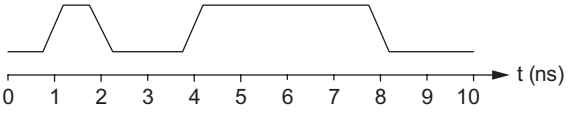


FIGURE 5.34 Signal for Exercise 5.4

5.5 Consider the buffer design problem from Example 4.14. If the delay constraint is 20 τ, how many stages will give the lowest energy, and how should the stages be sized?

5.6 Repeat Exercise 5.5 if the load is 500 rather than 64 and the delay constraint is 30 τ.

5.7 Derive the switching probabilities in Table 5.1.

5.8 Design an 8-input OR gate with a delay of under 4 FO4 inverters. Each input may present at most 1 unit of capacitance. The load capacitance is 16 units. If the input probabilities are 0.5, compute the switching probability at each node and size the circuit for minimum switching energy.

5.9 Construct a table similar to Table 5.2 for a 2-input NOR gate.

5.10 Design a header switch for a power gating circuit in a 65 nm process. Suppose the pMOS transistor has an ON resistance of about 2.5 $k\Omega \cdot \mu m$. The block being gated has an ON current of 100 mA. How wide must the header transistor be to cause less than a 2% increase in delay?